Project 4: A Frontend for zkDelphian

1 Introduction

The endpoint of Project 3 was zkDelphian, a zkSNARK for R1CS satisfiability. Recall the R1CS-sat relation: given matrices $A, B, C \in \mathbb{F}^{n \times n}$ and a public vector $\vec{x} \in \mathbb{F}^{n/2}$, there exists a witness $\vec{w} \in \mathbb{F}^{n/2}$ such that, defining $\vec{z} = (\vec{x} \parallel \vec{w})$, $A\vec{z} \circ B\vec{z} = C\vec{z}$, where $\circ$ is the Hadamard (entrywise) product. Each row of this equation is one *constraint*, and each entry of $\vec{z}$ is a *variable*. The matrices A, B, and C describe the computation to be proved.

In Project 3, you (and our tests) worked with A, B, C, $\vec{x}$, and $\vec{w}$ directly—constructing them by hand and passing them to the prover. This can be impractical for real computations. In this project you will build a *frontend*: a library that lets you describe a computation at a higher level of abstraction, and automatically translates it into the R1CS matrices and vectors that zkDelphian expects. Your frontend and gadgets will be written in Rust.

The frontend design you will use is typically called a *gadget library*. In this design, a developer writes *gadgets*—Rust functions that call two primitive operations, `alloc` (create a new variable in $\vec{z}$) and `enforce` (add a row to the matrices)—and the library handles everything else. A gadget can call other gadgets, so complex computations are built from small, testable pieces. In this way, the gadget library defines a kind of domain-specific programming language embedded in Rust.

Gadget libraries vs. compiler-based frontends. It is useful to contrast gadget libraries with another type of frontend design, a compiler-based frontend. In a compiler-based frontend, the developer writes something closer to ordinary imperative code, and a compiler transforms it into R1CS (or another constraint system). The developer does not call `alloc` or `enforce`; the compiler infers variables and constraints. This is more ergonomic but provides less control over constraint count and makes it harder to apply domain-specific optimizations; it also usually involves the compiler toolchain automatically synthesizing code for computing witnesses as opposed to the developer specifying this.

The goal. Your main task will be to implement gadgets for Boolean operations, 32-bit unsigned integer arithmetic, and a reduced-round variant of SHA-256. Your completed library will allow proving and verifying, using zkDelphian, knowledge of a 32-byte preimage of a given (reduced-round)SHA-256 hash.

1.1 Provided Code

The starter code consists of four files:

- `src/lib.rs`—the core library types, including `Variable`, `Circuit`, and `Matrixifier`, as well as the `ConstraintInterpreter` trait. You will implement the `Matrixifier` methods: `alloc`, `enforce`, `into_statement`, and `into_statement_and_witness`.
- `src/lc.rs`—the `Lc` (linear combination) type and its operator overloads.

- `src/snark.rs`—a wrapper that connects the frontend to the zkDelphian backend from Project 3.
- `src/gadgets.rs`—method stubs for the gadgets you will implement.

You will implement the function bodies in `src/lib.rs` and `src/gadgets.rs`. You should not need to modify the other files, but you are encouraged to read and understand them.

2 Core Library Types

This section explains the key types in the provided code. All of them map directly onto concepts from the matrix-vector formulation of R1CS we've discussed in class. A meta-point that is useful to keep in mind is that this library is meant to encode both the prover and verifier's view of the computation. This has some important implications that may be a bit subtle: for example, the need to distinguish between a public and private variable, and the fact that an allocated variable's constructor is given to both parties but when the constructor gets private variables as input it can only be evaluated by the prover.

2.1 `Variable<F>`

A `Variable<F>` represents a single entry in the vector $\vec{z}$. Each variable has two pieces of data:

- A **visibility**—`Public` or `Private`—which determines whether the variable belongs to the public input $\vec{x}$ or the private witness $\vec{w}$.
- An **index** within the public or private sub-vector.

The distinguished variable `Variable::one()` is always public index 0 and always holds the value 1. This is the standard R1CS convention: $\vec{z}[0] = 1$ always, which makes it possible to embed field constants into linear combinations. For example, the constant $7 \in \mathbb{F}$ is represented as $7 \cdot$ `Variable::one()`.

Note that `Variable<F>` has a phantom type parameter `F`. This exists only so that Rust's type system allows operator overloads like `Variable<F> + Variable<F> -> Lc<F>`. A `Variable` does not actually contain a field element.

2.2 `Lc<F>`: a linear combination of `Variables`

An `Lc<F>` is a linear combination of variables $c_1 v_1 + c_2 v_2 + \cdots + c_k v_k$ where each $c_i \in \mathbb{F}$ and each v_i is a `Variable`. Internally it is stored as a `Vec<(F, Variable<F>)>`—a list of (coefficient, variable) pairs.

In the matrix-vector picture, an `Lc` corresponds to **one row** of one of the matrices A, B, or C. The (coefficient, variable) pairs are exactly the nonzero entries of that sparse row. You can think of `Lc` as a sparse vector whose indices are `Variable`s rather than plain integers; the `Matrixifier` resolves these to integer column indices when it builds the final matrices.

The file `lc.rs` defines arithmetic operator overloads so that you can write linear combinations in near-mathematical notation inside gadgets:

- `var_a + var_b` produces the `Lc` with entries $[(1, a), (1, b)]$.
- `2 * var_a` produces $[(2, a)]$.
- `1 - var_a` produces $[(1, \texttt{one}), (-1, a)]$, since the integer `1` is interpreted as $1 \cdot$ `Variable::one()`.
- `var_a + var_b - var_c` produces $[(1, a), (1, b), (-1, c)]$.

2.3 `ConstraintInterpreter<F>`: The Gadget Interface

The trait `ConstraintInterpreter<F>` is the abstract interface that all gadgets program against. It has exactly two methods:

`alloc`. Creates a new variable (a new entry in $\vec{z}$). Arguments:

- An *annotation* (a string, for debugging), wrapped in a closure to avoid allocation when not needed.
- A `Visibility` (`Public` or `Private`).
- A *constructor* closure that computes the variable's concrete field value. It returns `Option<F>`: the prover supplies `Some(value)`, while the verifier may return `None` for private variables. This constructor is the way the programmer specifies how witness values are computed. (Recall in class we discussed a ZK frontend as outputting a compiled representation of a computation as well as a program for "witness extension"—the set of variable constructors is implicitly the witness extension program.)

The method returns a `Variable<F>`—a handle the gadget can use in later `alloc` calls and `enforce` calls.

`enforce`. Adds one R1CS constraint. It takes three arguments, each convertible to `Lc<F>`, representing the A-row, B-row, and C-row of the new constraint. The constraint asserts:

$$(\text{A-row} \cdot \vec{z}) \times (\text{B-row} \cdot \vec{z}) = (\text{C-row} \cdot \vec{z}).$$

Thanks to the `Into<Lc<F>>` conversions, you can pass a bare `Variable`, a `u64`, an `F`, or any expression that builds an `Lc`.

A design principle. Gadgets are useful because they allow decomposing complex programs into smaller pieces that can be reasoned about and tested individually. Towards this, an important principle of the design of our frontend is that gadgets are generic over `I: ConstraintInterpreter<F>`. A gadget does not know—or need to know—what other gadgets exist, or how other gadgets work internally. The ConstraintInterpreter just gives them an abstraction of a set of variables and constraints over them. Relatedly, a gadget does not need to know whether the interpreter is building matrices, checking satisfiability, counting constraints, or doing something else entirely.

2.4 `Circuit<F>`: The Top-Level Entry Point

The `Circuit<F>` trait has a single method:

```
fn synthesize<I: ConstraintInterpreter<F>>(
    self, cs: &mut I
) → Result<()>;
```

A type implementing `Circuit<F>` describes the *complete statement* to be proved. It is the top-level entry point that you hand to the `Matrixifier`. Only the outermost computation implements `Circuit`; internal building blocks are just functions (see §**??**).

2.5 `Matrixifier`: From Gadgets to Matrices

`Matrixifier<F, IS_PROVER>` is the concrete implementation of `ConstraintInterpreter` that builds the R1CS matrices. When `synthesize` runs against a `Matrixifier`:

- Each `alloc` call increments a variable counter and (for the prover, or for public variables) evaluates the constructor to obtain the concrete field element.
- Each `enforce` call stores three `Lc`s—one for each of A, B, C.

After synthesis, `into_statement()` converts the collected `Lc` rows into `SparseMatrix` objects. The column layout of the matrices is [public variables | private variables]: public variables occupy columns 0 through $n_x - 1$, and private variables occupy columns n_x through $n - 1$.

The type is parameterized by a const bool `IS_PROVER`. A `ProverMatrixifier` evaluates all constructors and produces both the statement $(A, B, C, \vec{x})$ and the witness $\vec{w}$. A `VerifierMatrixifier` skips private constructors (since the verifier does not know $\vec{w}$) and produces only the statement.

The connection to Project 3 is direct: `Matrixifier::into_statement()` produces exactly the types that zkDelphian's `prove` and `verify` functions expect.

3 Gadgets

3.1 What Is a Gadget?

A gadget is a function that takes `&mut impl ConstraintInterpreter<F>` (plus some inputs) and produces some outputs. It calls `alloc` to create new variables and `enforce` to add constraints.

Concretely, a gadget is a **Rust function** (or a method on a helper type), *not* a type that implements `Circuit`. `Circuit` is reserved for the top-level statement; gadgets are the building blocks that `Circuit::synthesize` calls internally. The hierarchy looks like this:

Top level: `struct PreimageProof` implements `Circuit<F>`
calls: `sha256(cs, input_bits)` — a gadget function
which calls: `sha256_compress(cs, state, block)` — a gadget function
which calls: `UInt32::add(cs, ...)` — a gadget method
which calls: `Boolean::xor(cs, ...)` — a gadget method

Every level shares the same `cs`. Variables accumulate in one $\vec{z}$; constraints accumulate in one set of matrices.

3.2 Recipe for Writing a Gadget

When you sit down to implement a new gadget, follow these steps:

1. **Write the relationship as a field equation.** What arithmetic identity, over $\mathbb{F}$, does the gadget need to enforce? For example, "a is a bit" becomes $a(1 - a) = 0$; "$c = a \oplus b$" (for bits) becomes $2ab = a + b - c$.
2. **Rearrange into the form (linear) $\times$ (linear) = (linear).** Each `enforce` call can handle exactly one multiplication of two linear combinations. If your relationship involves more than one multiplication, you must introduce *intermediate variables*—one `alloc` and one `enforce` per intermediate product.
3. **Decide what to allocate.** Each value that is not a linear combination of existing variables needs its own `alloc`. Outputs of the gadget are typically allocated here; intermediate values needed for multi-multiplication expressions are also allocated.

4. **Write the code.** Call `alloc` for each new variable. The constructor closure computes the concrete value from the gadget's inputs (for the prover). Then call `enforce` for each constraint, using the operator overloads to write the linear combinations readably.

5. **Return output handles.** Return the `Variable`s (or wrapper types like `Boolean`, `UInt32`) that represent the gadget's outputs. The caller uses these in subsequent gadgets.

Cost model. Every `alloc` adds one column to the matrices (one entry in $\vec{z}$). Every `enforce` adds one row. The cost of a gadget is (variables, constraints). Minimizing both is the main optimization challenge.

Purely linear operations are free (almost). If a relationship is purely linear—for example, $c = a + b$—you do not need a constraint at all. You can compute c's `Lc` as `var_a + var_b` and pass it directly to the next gadget. No `enforce` call, no new row. This is why some operations (like XOR of a variable with a constant, or bitwise NOT) can be zero-cost.

3.3 Gadget Composition: How Gadgets Call Other Gadgets

Gadgets compose because they all operate on the same `ConstraintInterpreter`. When gadget A calls gadget B with the same `&mut cs`:

- Variables allocated by B are added to the same $\vec{z}$ that A is building.
- Constraints added by B go into the same matrices.
- B returns `Variable` handles (or wrapper types) that A can use in its own `alloc` constructors and `enforce` calls.

This means you can build SHA-256 from `xor`, `and`, `add_mod32`, etc., without any special composition machinery. Each sub-gadget is a plain Rust function call.

3.4 `AllocatedBit` and `Boolean`: Bits in the Constraint System

One of the first things you will implement is a type representing a single bit. There are two types, for an important reason.

`AllocatedBit`. This is a wrapper around a `Variable<F>` that has been constrained to be binary via $a(1 - a) = 0$. It occupies one slot in $\vec{z}$ and costs one constraint. When you need a "real" bit backed by a variable—for instance, a bit of the prover's secret input—you use `AllocatedBit`.

`Boolean`. This is an enum:

```
enum Boolean<F> {
    Is(AllocatedBit<F>),    // the variable itself
    Not(AllocatedBit<F>),   // logical negation
    Constant(bool),         // a fixed true or false
}
```

The purpose of `Boolean` is to make certain operations *free*—that is, requiring zero additional variables and zero additional constraints:

- **Negation is free.** If you have a bit a, its negation is $1 - a$. You do not need to allocate a new variable b and constrain $b = 1 - a$. The `Not` variant records "this is the negation of that allocated bit," and when the value is used in an `enforce` call, the linear combination $1 - a$ is substituted automatically.
- **Constants are free.** If you XOR a variable with the constant `true`, the result is `Not(that_variable)`. If you AND anything with `false`, the result is `Constant(false)`. No constraint is needed.

These optimizations matter significantly for SHA-256, because the initial hash values and round constants are all fixed. Every operation involving a constant collapses, potentially saving thousands of constraints.

When you implement gadget methods on `Boolean` (such as `xor` and `and`), you will pattern-match on the variants and take the free path when possible, falling through to an `AllocatedBit`-level operation (which calls `alloc`/`enforce`) only when both operands are allocated bits.

3.5 `UInt32`: 32-Bit Words

SHA-256 operates on 32-bit words. A `UInt32` is represented as a little-endian vector of 32 `Boolean`s plus a cached `Option<u32>` value for the prover. Operations on `UInt32` decompose into operations on the individual bits. For example:

- **Bitwise XOR, AND, NOT** apply the corresponding `Boolean` operation to each bit position independently.
- **Rotation and shift by a constant** simply rearrange (and possibly zero out) the bit vector. These are completely free: no variables, no constraints.
- **Addition modulo** 2^{32} is the expensive operation. You must constrain a carry chain.

An annotated example: XOR. You will find an annotated example of an XOR gadget in `TODO`. It is an implementation of the `Circuit<F>` trait.

4 Your Task

Your goal is to implement all the gadgets in `src/gadgets.rs` and a top-level `Circuit<F>` implementation for the relation

$$\mathcal{R} = \{(h, m) \,:\, \mathsf{SHA256}_4(m) = h \wedge |m| = 32\}$$

where $\mathsf{SHA256}_4$ is a reduced-round variant of SHA-256 that uses only four rounds instead of the full 64. A good discussion and specification of the SHA-256 hash function can be found on the Wikipedia page for SHA-2.

4.1 Implementation Order

We recommend implementing the gadgets bottom-up:

1. `AllocatedBit`—`alloc`, and the booleanness constraint.
2. `Boolean`—the enum, plus `xor`, `and`, `not`, with constant folding.
3. `UInt32`—from bits, rotation, shift, bitwise ops, addition mod 2^{32}.

4. SHA-256 round function—`ch`, `maj`, Σ_0, Σ_1, σ_0, σ_1, and the compression function.

5. Message schedule (the W_t expansion).

6. Top-level `Circuit` that allocates input bits, calls SHA-256, and constrains the output to match the public hash.

4.2 Connecting to zkDelphian

The file `src/snark.rs` provides a wrapper that takes your `Circuit`, runs it through a `ProverMatrixifier` (or `VerifierMatrixifier`), and passes the resulting matrices and vectors to the zkDelphian prover and verifier from Project 3. You should not need to modify this file.

Running tests. As with previous projects, always run tests with `cargo test -release`.

5 Submission Instructions and Grading Criteria

To submit your project, first [XXX] then run the command

```
zip submission.zip -r p1/src/ p2/src p3/src p4/src -x '*/target/*'
```

in the root of p4. Then, submit the resulting `submission.zip` file to autograder.io.

Autograder.io will run our test suite using your submitted code and compute your grade. Your grade will be the percentage of tests that pass. Any modifications you make to the `tests/` directory will not impact how your code is graded. (Feel free to add your own tests locally; this can be a good way to test your own understanding.) Note that doing something "fishy" to try to get a better grade—for example, overwriting any equality methods—will result in failing the assignment.